

DEGREE PROJECT IN COMPUTER SCIENCE AND ENGINEERING, SECOND CYCLE, 30 CREDITS

STOCKHOLM, SWEDEN 2020

OpenMZ: a C implementation of the MultiZone API

HENRIK KARLSSON

KTH ROYAL INSTITUTE OF TECHNOLOGY
SCHOOL OF ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

OpenMZ: a C implementation of the MultiZone API

HENRIK KARLSSON

Master in Computer Science
Date: 2020/06/30
Supervisor: Roberto Guanciale & Didrik Lundberg
Examiner: Mads Dam
School of Electrical Engineering and Computer Science
Host company: Saab AB
Swedish title: OpenMZ: en C-implementation av MultiZone API:et

Abstract

We implemented, benchmarked, and analyzed OpenMZ, a separation kernel for RISC-V targeting secure coprocessors and embedded devices. OpenMZ is an open-source implementation of the MultiZone API, which partitions a system into a fixed number of zones that can communicate with each other and handle interrupts.

OpenMZ has a small code base of around 800 lines, making the kernel formally verifiable, and with an open-source RISC-V processor, it can be the foundation of a fully verified computing system. Moreover, OpenMZ has some real-time properties and a context switch of around 500 cycles, making it suitable for real-time applications.

Our security analysis shows that OpenMZ protects the zones' memory from direct access with RISC-V's PMP-mechanism, but not indirect access, e.g., through the DMA. Moreover, the MultiZone API has some vulnerabilities exposing the zones to both DoS attacks and timing side-channel attacks. To address these security issues, we suggest a few modifications to the API and the scheduler.

Sammanfattning

Vi implementerade, benchmarkade och analyserade OpenMZ, en separationskärna för RISC-V som riktar sig till säkra hjälpprocessorer och inbyggda system. OpenMZ är en öppen implementation av MultiZone API:et vilket delar upp ett system i ett fixt antal zoner som kan kommunicera med varandra och hantera avbrott.

OpenMZ har en liten kodbas på cirka 800 rader vilket gör kärnan formellt verifierbar, och med en open-source RISC-V-processor, kan den vara grunden till ett fullständigt verifierat datorsystem. OpenMZ har dessutom vissa realtidsegenskaper och ett processorbyte på cirka 500 cykler, vilket gör den lämpligt för vissa realtidsapplikationer.

Vår säkerhetsanalys visar att OpenMZ skyddar zonernas minne från direkt åtkomst med hjälp av RISC-Vs PMP-mekanism, men inte indirekt åtkomst, t.ex. genom DMA:et. Dessutom har MultiZone API:et sårbarheter som utsätter zonerna för både DoS-attacker och timing-attacker. För att åtgärda dessa säkerhetsproblem föreslår vi därför några ändringar av API:et och schemaläggaren.

Contents

1 Introduction **1**

2 Background **3**
2.1 RISC-V . . . 3
2.1.1 Registers . . . 4
2.1.2 Interrupt and Exception Handling . . . 5
2.1.3 Physical Memory Protection . . . 8
2.2 Trusted Execution Environment . . . 9
2.2.1 ARM TrustZone . . . 9
2.2.2 Intel SGX . . . 10
2.2.3 Keystone . . . 12
2.2.4 MultiZone Security . . . 13
2.3 Security Kernels . . . 15
2.4 Side-Channel Attacks . . . 19
2.5 Secure Interrupts . . . 20
2.6 CompCert . . . 20

3 Methods **22**
3.1 Design . . . 22
3.2 Attacker Models . . . 23
3.3 Size Benchmark . . . 23
3.4 Performance Benchmark . . . 24

4 Results **27**
4.1 Implementation . . . 27
4.2 Size Benchmark Results . . . 32
4.3 Performance Benchmark . . . 33
4.4 Real-Time Properties . . . 34
4.5 Security Analysis . . . 36
4.5.1 Data-Based Attacks . . . 36

4.5.2 Timing Vulnerabilities 37

5 Discussion **39**

6 Conclusion **42**

Bibliography **46**

A RISC-V **50**

B MultiZone API **53**

Chapter 1

Introduction

The security and reliability of a computer system are based on both the software and the hardware. The most privileged software is the kernel, fully controlling both the hardware and the less privileged software. Therefore, any bug in the kernel can compromise the entire platform, so ensuring that the kernel is bug-free is critical for any security and safety-critical application. A bug-free kernel is, however, not sufficient.

In 2017, researchers discovered serious security vulnerabilities in several proprietary CPUs. One of those vulnerabilities is Meltdown [1] that enables rogue processes to read the main memory of a system freely. The Meltdown vulnerability exists mainly in Intel processors from 1995 to 2017. We can fully patch Meltdown through firmware, but we can not patch Spectre [2], a related vulnerability that enables data leaks through branch mispredictions on pre-2019 processors. Fortunately, it is hard to exploit Spectre, and we have mitigations making it even harder to exploit. These two vulnerabilities do, however, pose the question, "Can we trust proprietary hardware?"

RISC-V [3] is an open-source RISC ISA that is quickly gaining traction in both academia and industry. RISC-V's open-source license allows anyone to implement the ISA, which in turn allows customers to choose a RISC-V manufacturer. The ability to choose a manufacturer is important for both safety- and security-critical applications since the manufacturer and the customer may have conflicts of interest. Although the ISA does not prohibit closed-source implementations, there are open-source implementations of RISC-V. These open-source implementations allow anyone to analyze them, find bugs and know that the processor is bug-free.

A bug-free implementation of RISC-V, together with a suitable security kernel, can then be the foundation of a fully verified system.

HexFive's MultiZone Security (MZS) is a proprietary security solution for RISC-V [4]. The solution includes a security kernel and a secure bootloader. The security kernel partitions a system into several spatially isolated zones and schedules them with either a round-robin scheduler or a cooperative scheduler. The kernel supports, among other things, interrupt and exception handling in zones, inter-zone messaging, and timer interrupts for zones. The secure bootloader ensures that the system runs the MZS kernel with only trusted software. Although MZS is proprietary, it is of interest to fully verified systems because the MultiZone API is open-source, allowing us to implement an open-source version of the kernel.

Thesis objective. For this thesis, we implement, benchmark, and analyze OpenMZ, an open-source version of the MultiZone Security kernel. The target applications are secure coprocessors and secure embedded devices. We set and attempt to fulfill the following design goals: verifiability, small memory footprint, high-security, real-time properties, and high-performance. The precise definition of these goals and how we attempt to fulfill them is described in Section 3.1. Our problem statement is thus, "How well does OpenMZ achieve these design goals?"

Social and ethical impact. A verified security kernel has both positive and negative social and ethical impacts. The immediately obvious impacts are that of increased system security and transparency, allowing us to trust our computer systems to a higher degree. This creates a possibility for system manufacturers to create fully verified products, allowing them to increase customer trust or eliminate the need for trust. A manufacturer can then implement, say, a transparent and verified system monitor without worrying the customer. We can also choose to hide the system monitor from the user if the security kernel is perfect, then the user has no possibility of knowing what is happening in their system. This can be considered both positive and negative, depending on who is monitoring and who is being monitored.

Chapter 2

Background

In this chapter, we present the background necessary to understand the OpenMZ kernel, its relation to the state-of-the-art security kernels, and its limitations. We start by introducing the RISC-V ISA since the OpenMZ kernel uses RISC-V features. After that, we present the concept of trusted execution environments, which is intimately related to security kernels, and then we introduce the concept of security kernels. For the security analysis of the OpenMZ kernel, we present side-channel attacks and secure interrupts. Finally, we present CompCert, the formally verified C compiler, which is related to OpenMZ's formal verifiability.

2.1 RISC-V

RISC-V is an open-source RISC ISA developed in U.C. Berkeley [3, 5, 6]. In 2010, Krste Asanović, Yunsup Lee, and Andrew Waterman initiated the development of RISC-V, releasing the first version of the ISA in 2011. The main feature of RISC-V is its open-source license that allows anyone to produce a RISC-V processor, but this is not the only feature. There have been other open-source ISAs before RISC-V (OpenRISC, SPARC), but most of these have failed due to commercial or technical reasons [7]. Learning from the mistakes of these earlier ISAs, Asanović, et al. design RISC-V to be technically superior by fixing the issues of earlier ISAs, and support RISC-V commercially by founding the RISC-V International foundation, and SiFive, a company that produces RISC-V IP (e.g., RISC-V processor designs).

Today, RISC-V has three different privilege modes; machine mode, supervisor mode, and user mode. The machine mode (M-mode) has

the highest privilege level and can access all system resources. The supervisor mode (S-mode) is for UNIX-style operating systems. Finally, the user mode (U-mode) is meant for user-level applications. The RISC-V developers are currently working on a fourth privilege mode, the hypervisor mode (H-mode), meant for hypervisors, sitting in between the machine and supervisor modes.

RISC-V is designed around extensions, making it suitable for many different applications. Due to the extensions, RISC-V is not an ISA in the classical sense, but more of a family of ISAs. RISC-V has four base ISAs, RV32I, RV32E, RV64I, and RV128I. The RVxI ISAs have a base set of 32 x-bit general-purpose integer registers with basic integer arithmetic, comparisons, unconditional/conditional jumps, and memory load/store instructions. RV32E is the same as RV32I but with only 16 general-purpose registers.

We can combine the base ISA with several instruction and privilege extensions, making the ISA very flexible. For example, the M extensions add integer multiplication and division, while the S extension adds the supervisor mode. In Table A.1 in the Appendix A, we have a list of all the currently standardized or developing extensions. We denote a RISC-V ISA by appending the extension names to the base ISA. For example, RV32IMAC is the RV32I base ISA with integer multiplication and division (M), atomic instructions (A), and compressed instructions (C).

RISC-V is not only flexible when it comes to extensions but also the composition of different types of RISC-V cores. RISC-V supports processors with multiple hardware threads (harts) and heterogeneous cores. For example, we can have four cores with two harts, each supporting M, S, and U-mode, and in the same CPU. Moreover, we can have an additional core with only M and U-mode, working as a secure coprocessor to the primary system.

2.1.1 Registers

There are three types of registers in RISC-V; general-purpose registers, control and status registers (CSRs), and memory-mapped registers.

As mentioned earlier, RISC-V has by default 32 general-purpose integer registers `x0-x31` per hart. Register `x0` is hard-wired to zero, enabling some optimizations and instruction overloading. Registers `x1-x4` has the return address, global, stack, and thread pointer, respectively. Table A.2

in Appendix A shows the temporary, saved and argument registers. With the floating-point extensions, we have an additional 32 general-purpose floating-point registers `f0-f31` for each hart.

RISC-V supports up to 4096 CSRs, but most of these are reserved for future use or customization. All the CSRs can be read, written, set, or cleared atomically using special CSR instructions. RISC-V uses these registers mainly for interrupt/exception handling and setup, machine information, counters, memory protection, and virtual memory management. All CSRs are per-hart registers, meaning that a hart can run an almost entirely independent context.

RISC-V also has a set of memory-mapped registers. For example, the PLIC registers (introduced in the next section), and the timer registers are memory-mapped. Several harts naturally share these registers since the harts share the system's resources.

2.1.2 Interrupt and Exception Handling

In RISC-V, we collectively refer to interrupts and exceptions as traps. Traps are by default handled in M-mode, but we can configure the hardware to delegate traps to the next-lower privilege level such as S-mode, or U-mode if we have the N-extension (User-Level Interrupts) and no S-mode. Moreover, traps delegated to S-mode can be further delegated to U-mode if we have the N-extension.

Registers. RISC-V has six trap setup registers and five trap handling registers in M-mode, all of which are CSRs. Most of these registers have a corresponding variant in S-mode and U-mode. As the names suggest, we use the trap setup registers to set up the trap handler, and we use the trap handling registers during the trap handling process.

Trap Setup Registers. The most important register in RISC-V is `mstatus` which controls and keeps track of the hart's current operating state. The `mstatus` register includes, among other fields, the following:

- `mstatus.mpp` (machine previous privilege), privilege mode before the last trap,
- `mstatus.mie` (machine interrupt enabled), enables/disables interrupt globally, and
- `mstatus.mpie` (machine previous interrupt enabled), the value of `mstatus.mie` before the latest trap was triggered.

The `mie` register (not to be confused with `mstatus.mie`) enable and disable individual interrupts. Each bit in the `mie` register controls one interrupt, when the bit is set, the interrupt is enabled. The standard M-mode interrupts are machine software interrupt (inter-processor interrupt), machine timer interrupt, and machine external interrupt (platform-level interrupt). RISC-V implementations may have corresponding interrupts for lower privilege modes (e.g., supervisor timer interrupt), allowing an M-mode kernel to delegate interrupts.

The `mtvec` register holds the trap handler's base address and mode. The trap handler can have two modes: direct mode, causing traps to jump to the base address; and vectored mode, treating the base address as the base of a branch table. In RISC-V, each trap has an *exception code* identifying the event that caused the trap. In vectored mode, all exceptions jump to the base address, while interrupts jump to the base address plus four times the exception code.

It is the `medeleg` and `mideleg` registers that allow us to delegate exceptions and interrupts respectively to the next-lower privilege mode. Each bit of `medeleg` and `mideleg` controls the delegation one trap. If a bit is set, the corresponding trap is delegated to the next-lower privilege mode, e.g., S-mode, so when the trap is triggered, it is handled directly in S-mode. Note that traps never transitions to less-privileged mode; that is, traps taken in M-mode are never delegated to S-mode.

To allow lower privilege modes to measure performance, we have the `mcounteren` register, which permits the next-lower privilege mode to read M-mode performance counter and timer registers.

Trap handling registers. The five trap handling registers are `mepc`, `mcause`, `mtval`, `mscratch`, and `mip`. We describe these registers by how we can use them on a system running a U-mode program with an M-mode kernel. Suppose a trap is triggered while the system runs in U-mode, then the hardware does the following:

1. sets `mepc` to `pc`,
2. sets `pc` to `mtvec` (assuming direct mode on `mtvec`),
3. writes the exception code to `mcause`,
4. writes zero or auxiliary exception information to `mtval`,
5. saves the interrupt state (`mstatus.pie = mstatus.ie`),
6. disables interrupts globally (`mstatus.ie = 0`),
7. sets the previous privilege to U-mode (`mstatus.mpp = 0`), and
8. sets the current privilege mode to M-mode.

The `pc` now points at the kernel's trap entry function which starts by swapping a user `sp` with the kernel `sp` in `mscratch`. After the swap, we store the U-mode context to the kernel stack. At this point, the kernel is ready to handle the trap, so it reads the `mcause` register to identify the trap cause and invoke the correct handler.

Once the kernel has handled the trap, the kernel repeatedly reads the `mip` register as it keeps track of interrupts that are awaiting handling. If `mip` is non-zero, we have a pending interrupt, so the kernel identifies the interrupt, handles it, and then reads `mip` again. If `mip` is zero, all interrupts have been handled, so the kernel restores the user context and executes the `mret` instruction, which we describe below.

Trap instructions. RISC-V has a few instructions related to traps, `ecall`, `ebreak`, `mret`, `sret`, and `uret`.

RISC-V uses the `ecall` instruction to request services from the execution environment (e.g., a kernel). The `ebreak` instruction stops execution and trigger an external debugger, but can also be used as an alternative to `ecall`.

The *x*`ret` instructions trigger a trap return which sets the `pc` to *x*`epc`, *x*`status.ie` to *x*`status.`*x*`pie`, and the privilege level to *x*`status.`*x*`pp`.

Interrupt types. RISC-V has two types of interrupts, core-local interrupts (CLI), and platform-level interrupts (PLI). As the names suggest, PLIs are shared by all the harts of a system, while the CLIs are per-hart (per-core-thread). The PLIs and CLIs are designed to complement each other; in a multi-core system, the harts share the PLIs and have one set of CLIs each. In a single-hart system, we might only have CLIs.

Platform-level interrupts. Because PLIs are platform-wide, we need the PLI controller (PLIC), which distributes platform-level interrupts in a RISC-V system, preventing two or more harts from handling the same interrupt. We interact with the PLIC through a set of memory-mapped registers, and through it, we can register interrupt targets (e.g., harts) to interrupt sources (e.g., I/O devices), and set the priorities of these interrupt sources. We can allow PLIC to interrupt a hart by setting the "machine external interrupt" bit on the `mie` register, and we can check if any PLIs are pending by checking the same bit in the `mip` register.

When the PLIC receives an interrupt request, it will forwards the interrupt to the registered targets, which can then attempt to claim it

from the PLIC. The target that manages to claim the interrupt will then handle it and signal the PLIC upon its completion.

Core-local interrupts. Each CLI has its own exception code, unlike the PLIs which share the "machine external interrupt" exception code. We can, therefore, quickly identify the cause of the CLIs by reading the `mcause` register, and we can vector their CLI handlers using `mtvec`'s vectored mode. We can enable and disable individual CLIs using the `mie` register, and check if there are pending interrupts with the `mip` register. The standard RISC-V CLIs are the machine timer interrupt, machine software interrupt, and their S- and U-mode counterparts.

2.1.3 Physical Memory Protection

The physical memory protection (PMP) unit provides per-hart M-mode only CSRs that control the memory access privileges for up to 16 memory regions with a 4-byte granularity. PMP checks are always applied to harts running in S- or U-mode, and optionally on M-mode. In short, PMP can grant permissions to S- and U-mode processes, which by default have none, and can revoke permissions from M-mode processes, which have full permissions by default.

The PMP registers include up to 16 pairs of 8-bit-wide PMP configurations (`pmp`*i*`cfg`) and PMP addresses (`pmpaddr`*i*), Figure A.1 in the appendix shows the layout of the registers.

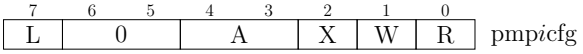


Figure 2.1: PMP configuration format.

In figures 2.1, we have the format of the PMP configuration registers. The R/W/X bits grant read/write/execute permissions to the U- and S-mode processes. The L bit locks the PMP configuration as well as the related address registers, and it enforces the PMP configuration on the M-mode processes. Moreover, we can only clear the L bit with a system reset.

The A field encodes the address matching mode, and we can set it to the following:

- **Off.** Disables the PMP register.
- **Top-of-the-region (TOR).** The corresponding address, `pmpaddr`i, is the top of the region, and the previous address `pmpaddr`$(i-1)$ is the bottom of the regions. If $i = 0$, the bottom of the region is 0.
- **Naturally-aligned-4 (NA4).** The region has the size of 4 bytes, and the base address `pmpaddr`i is a multiple of 4.
- **Naturally-aligned-power-of-two (NAPOT).** The region has the size 2^G where $G > 2$, and the base address `pmpaddr`i is a multiple of the size. Table A.3 in Appendix A shows how the size of the region is encoded in `pmpaddr`i.

2.2 Trusted Execution Environment

A *trusted execution environment* (TEE) is a protected part of the processor that accepts only trusted software, and can remotely attest its trustworthiness to a third-party [8]. TEEs are often designed to run in parallel to a rich execution environment (REE, environment running conventional OSs), providing secure functions to a user-facing OS and its applications.

2.2.1 ARM TrustZone

ARM TrustZone [9] is a collection of hardware modifications that partitions a system's resources into a secure world (i.e., TEE) and a normal world (i.e., REE). In the secure world, we have a trusted OS that manages the partitioning of the secure and the normal world as well as performs trusted functions. In the normal world, we have a normal OS, such as Android. In figure 2.2, we have an illustration of the system partitioning.

TrustZone partitions on a system level so we can assign both memory regions and peripherals to the secure world. TrustZone prevents the normal world components from accessing the secure world using hardware mechanisms but does not prevent the secure world from accessing the normal world. The secure world can support a full operating system but

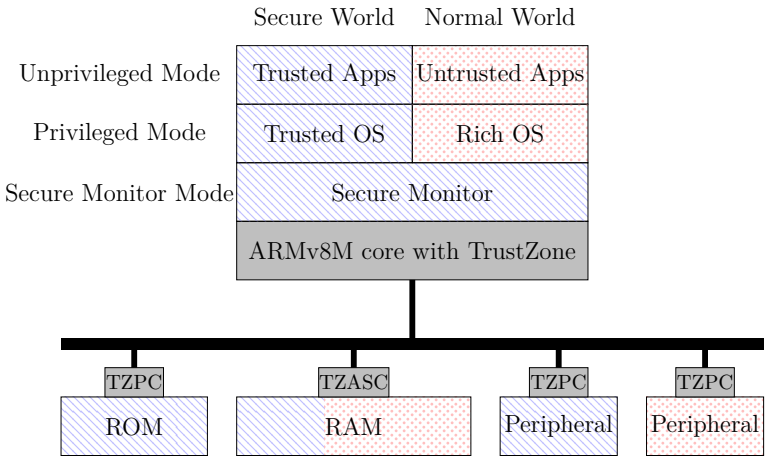


Figure 2.2: The secure world has full access to the system by default and grants access to the normal world. Every external and internal component must be adapted to TruztZone using adapters and modules such as the TrustZone Peripheral Controller (TZPC).

should contain a trusted OS implementing the so-called GlobalPlatform TEE standard, for example, OP-TEE [10].

Besides the trusted OS, the software TCB consists of a secure monitor and secure bootloader. The secure monitor has the highest privilege level (EL3) and implements a mechanism for context switching between the secure and normal world. The secure bootloader is responsible for bootstrapping the system into a secure state. Note that the TrustZone modifications do not protect against physical attacks by default (e.g., replacing the ROM), so the implementer must appropriately secure the boot code.

2.2.2 Intel SGX

Intel Software Guard Extension (SGX) is a TEE for executing sensitive software on a remote and untrusted computer using trusted hardware [11]. SGX can protect both code and data, guarantee enclave integrity, and provide both local and remote attestation. Intel SGX can only assign a contiguous memory region to each enclave, so we can only use enclaves for secure computations since they can not access peripherals.

In SGX, an enclave has the same privilege level as a user process and sits in a user process's address space (see figure 2.3). This placement means that the enclave obeys the same address translation rules as a

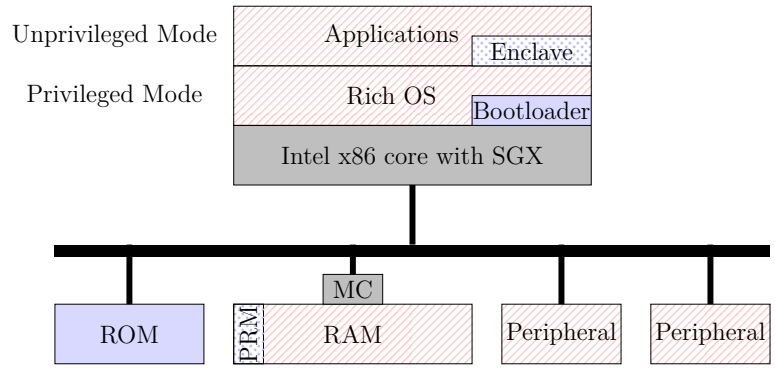


Figure 2.3: SGX enclaves reside logically inside applications but physically in the PRM region of the memory. The memory controller (MC) ensures that non-enclave processes can not access the enclave regions, and it encrypts the enclave memory. Enclaves have no access to peripherals, so they are for secure computations only.

usual process; that is, the host OS manages its page tables. SGX is, therefore, fully compatible with existing memory layouts, and the MMU can fully manage it. SGX has some restrictions on enclaves. For example, an enclave is not allowed to trap into the OS, and it must reside in a relatively small memory region called the Processor Reserved Memory (PRM).

The CPU protects all non-enclave accesses to the PRM region. The PRM region contains the enclaves pages (i.e., code and data) and metadata. If enclave data exceeds the capacity of the PRM, the OS can write the data outside the PRM region. This data is transparently encrypted by the memory controller to protect it from the OS.

Enclaves are created and initialized by untrusted processes. Once the untrusted processes have initialized the enclave, the hardware ensures that the untrusted processes can not modify the enclave data. During the initialization, the CPU measures the code and the static data. This measurement is then the basis for attestation. All operations involved in the management of an enclave, except interrupts, is performed by the host OS. If an interrupt occurs while running an enclave, the CPU performs an asynchronous enclave exit (AEX), which results in a secure context switch, preventing information leakage to the untrusted system.

2.2.3 Keystone

Keystone is a configurable TEE for the RISC-V ISA with configurable enclaves [12]. Like SGX enclaves, Keystone enclaves are meant for secure computations only; they do not have access to peripherals. Keystone does, however, support smaller supervisors in the enclaves, managing enclave resources.

In figure 2.4, we have three types of trusted software, the security monitor (SM), the enclave runtime (RT), and the enclave application (eapp). The SM is a minimal kernel that can create, destroy, and authenticate enclaves on behalf of the rich OS. The enclave in Keystone consists of an RT and an eapp. The RT is an SM-aware supervisor that provides system services such as secure memory management and enclave-host communication to an eapp. The eapp is simply a trusted application.

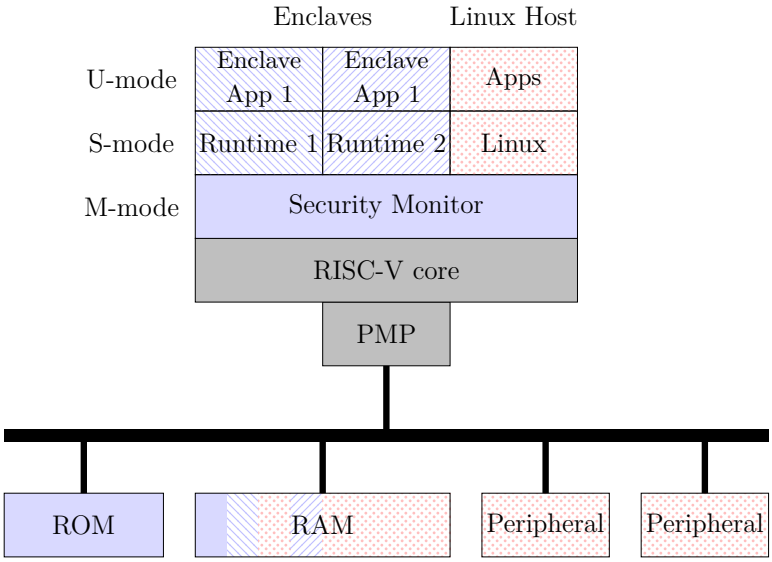


Figure 2.4: A Keystone enclave consists of an enclave application and a supervisor (i.e., runtime) providing services to the application. The security monitor protects the partitions, manages the enclaves, and provides services for both the enclaves and the host. Each enclave has access to one region of the RAM, allowing it to do secure computations.

Keystone protects enclaves using RISC-V's PMP mechanism, where each enclave is by default given a fixed contiguous memory region. We can enable flexible memory management of the enclave by using various plugins such as the free-memory plugin, which allows an enclave to reserve memory without mapping it at creation time. Another plugin

is the edge call interface that enables enclave-host communication using shared memory.

The security of Keystone relies on the SM, the RT, the bootloader, and the cryptographic library. The SM is responsible for ensuring that the memory of the enclaves and the host OS is protected, but it does not protect them against timing attacks [12]. The RT is responsible for managing the eapp memory and service the eapp. Finally, the bootloader and the cryptographic library are responsible for loading an authorized SM and creating attestations of the system's trustworthiness. The trusted computing base of Keystone ends up at 12-15 KLoC, making it suitable for formal verification.

2.2.4 MultiZone Security

MultiZone Security (MZS) by HexFive is a minimal proprietary TEE for RISC-V that partitions a core into a fixed number of equally protected zones running in U-mode [4, 13]. The MZS TEE consists of the MultiZone kernel (nanoKernel) running in M-mode, and a secure bootloader, verifying the trustworthiness of the software running on the platform. The kernel was written in pure assembly, resulting in a minuscule kernel overhead, and making it suitable for real-time applications, embedded devices, and secure coprocessor. In figure 2.5, we have the kernel on the left-hand side acting as a secure coprocessor to a Linux companion on the right-hand side. Note that the Linux companion is optional and that we can run MultiZone independently.

The MZS kernel schedules the zones with a round-robin or cooperative scheme. To support the cooperative scheme, the MultiZone API includes the `ECALL_YIELD()`, and `ECALL_WFI()` system calls. The `ECALL_YIELD()` system call causes the kernel to bring the next zone into context. The `ECALL_WFI()` hints to the scheduler that the zone is waiting for an interrupt (WFI). The kernel can then pause a zone until an interrupt wakes it. If all zones are waiting, the kernel can suspend the core. Since the WFI is just a hint, it is legal to implement `ECALL_WFI()` as `ECALL_YIELD()`, in which the core is never suspended.

The MZS kernel isolates the zones using RISC-V's PMP mechanism and provides up to six memory-mapped regions for each zone. This PMP utilization means that zones have separate code and data regions as well as up to four memory-mapped I/O regions. MZS zones can, therefore,

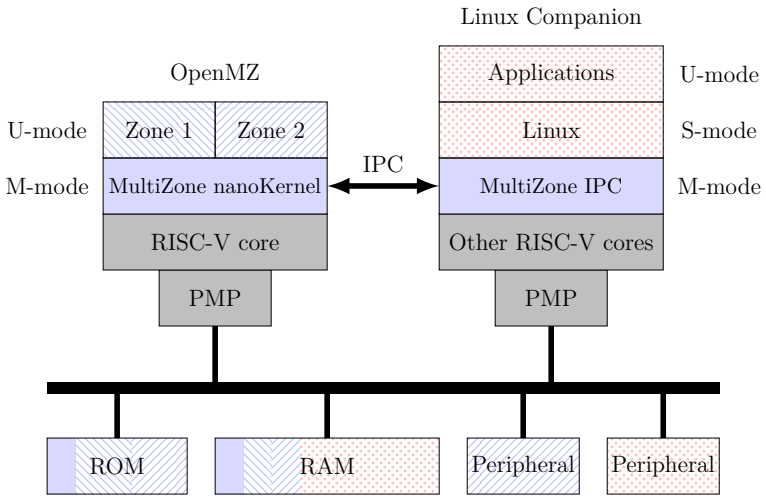


Figure 2.5: The MultiZone nanoKernel partitions a system into a fixed number of zones running in U-mode and provides services to them. Each zone has up to six memory-mapped regions. MultiZone has an optional Linux component, allowing MultiZone to run alongside a Linux host and provide services to it. MultiZone includes a secure bootloader, ensuring that only trusted software runs on the platform.

handle I/O devices. The PMP regions of the MZS zones are allowed to overlap, but this comes at the cost of memory isolation.

The MultiZone API includes two non-blocking system calls for secure inter-zone communication, `ECALL_SEND(receiver id, int[4])` for sending messages, and `ECALL_RECV(sender id, int[4])` for receiving messages. Messages are 16-bytes long, and they are transmitted through the kernel using registers. Each zone has inboxes with one entry for every other zone, allowing them to receive one message at the time from each zone.

The zones can also communicate with the optional Linux companion using `ECALL_SEND` and `ECALL_RECV`, in which case the Linux companion is identified as receiver or sender 0. The Linux companion and MultiZone communicate through a shared buffer split into a MultiZone inbox and a Linux inbox, each of which has one entry for every zone. The MultiZone inbox is used for sending messages from the Linux companion to the zones, while the Linux inbox is similarly used for the other direction.

MZS has full support for interrupt handling in U-mode without RISC-V's N-extension. We assign interrupts to zones during kernel configuration. A zone can register handlers for the interrupts that we have assigned to it using the `ECALL_IRQ_VECT(interrupt code, handler)` system call. When

an interrupt occurs, the kernel brings the zone assigned to that interrupt into context and executes the registered interrupt handler. A zone can globally disable and enable its interrupts with the `ECALL_CSRC_MIE()` and `ECALL_CSRS_MIE()` respectively.

MZS also has full support for exception handling in U-mode. A zone can register an exception handler for every type of exception available using the `ECALL_TRP_VECT(exception code, handler)` system call. When a zone triggers an exception, the zone executes the corresponding exception handler. MZS also has secure timer interrupts, which are registered with exception code 3, the zone can then set a timer with `ECALL_CSRW_MTIMECMP(timeout)`. This timer is a soft timer, only enabled when the zone is in context.

Lastly, MZS has system calls for reading M-mode CSRs such as `ECALL_CSRR_MCYCLE()` that returns the value of the `mcycle` CSR. MZS also emulates many CSR reads that would normally be illegal in U-mode. Executing a CSR read system call is, however, preferable since it is more performant.

HexFive have open-sourced the MultiZone API [14], encouraging others to implement it, and allowing us to create an implementation that we can formally verify. In Appendix B, we have a full description of the MultiZone API.

2.3 Security Kernels

In this thesis, we define a security kernel to be any kernel where the security and verification of the kernel are prioritized.

Reference monitor. In 1972, Anderson [15] introduced the first security kernel as the reference monitor. The reference monitor enforces an access policy on a multi-user system, satisfying three requirements that Anderson had identified for a secure system: access control, authorization, and controlled executions of both kernel and user programs.

Today, the reference monitor is central to the security of most monolithic OSs such as Linux [16], and Windows NT [17]. Verification of these types of kernels has, however, met difficulties [18]. The difficulties stem mostly from the coupling of the trusted functions and the kernel, making the kernel large and mutable. The size of the kernel makes verification very difficult, but the mutability makes it impossible as when we add new drivers or functions, we undo the verification. Moreover, the reference

monitor concept is not suited for more specialized systems whose security requirements are often different from multi-user systems. Due to these problems, Rushby [18] introduced the concept of the separation kernel.

Separation kernel. The separation kernel is a security kernel that partitions a system into isolated environments that can only communicate with each other when explicitly permitted [18]. When formulating the concept of the security kernel, Rushby was inspired by the security of distributed systems, which is achieved by the physical separation of components and trusted functions. Thus, we realize the separation kernel by emulating a distributed system on a single processor. This approach completely decouples the verification of individual components from the verification of the kernel, making verification of the kernel feasible and worthwhile.

Rushby [19] also introduced the "proof of separability" technique, which we can use to prove that a separation kernel emulates a distributed environment, thus proving its security. He formulated six conditions for this proof [19], which we can informally summarize as "the state change of a partition or an I/O device depend only on the previous state and the input". In figure 2.6, we illustrate these conditions with two partitions, red and blue. These conditions do, however, exclude the explicitly permitted communications channel between partitions. To allow communication between partitions, Rushby relaxes his conditions for a special WRITE operation, but others have also formulated the communications channels as an I/O device to circumvent the conditions [20, 21, 22]. Moreover, the proof model does not consider covert channels, e.g., timing channels, as Rushby dismisses them as having too low bandwidth and being easily counteracted with noise.

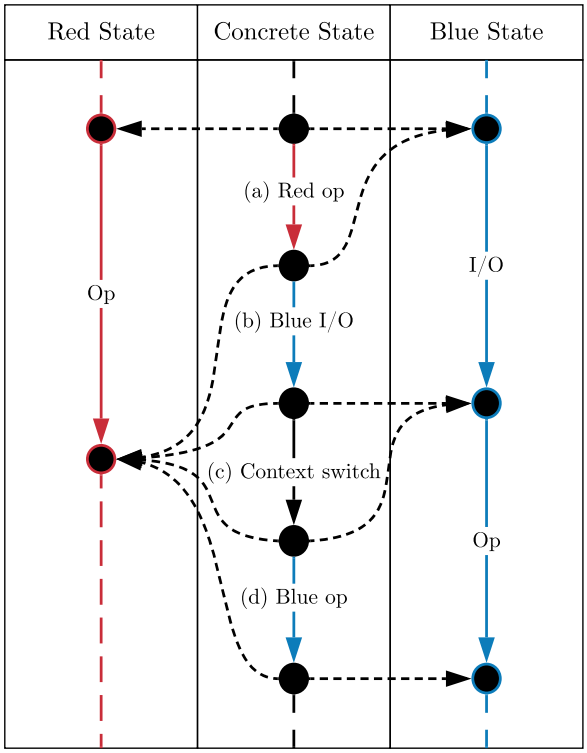


Figure 2.6: In the center, we have the concrete (i.e., host) state. On the left and the right, we have the states of the red and blue partitions, respectively. The solid arrows are state transitions. The dotted arrows represent blue and red partitions abstraction or view of the concrete state. *(a) The host emulates red's operations while leaving blue unaffected. (b) Blue's I/O device affects the concrete state and the blue state but not the red state. (c) The host's context switching does not affect the partitions. (d) The host emulates blue's operations while leaving red unaffected.*

INTEGRITY-178B by Green Hills is a proprietary RTOS with a separation kernel [22]. INTEGRITY-178B has a fixed number of spatially and temporally isolated partitions. A partition in INTEGRITY-178B can support an OS so that a machine can run a user interface on Linux or Windows while simultaneously managing safety-critical applications. Green Hills designed the OS for avionics, and since these applications are

both safety- and security-critical, they semiformally verified [1] and tested the OS according to EAL6 [23].

Microkernel. In a *microkernel OS*, we minimize the kernel by placing most system services in the userspace [24]. The kernel concentrates on supporting processes in the form of virtual memory management, scheduling, and communication between processes. All other OS functionalities, such as hardware and window management, are done by userspace processes. By placing system services in userspace, it is easier to extend the OS since we do not have to modify the kernel. The OS should also be more portable since the kernel is small and has limited functionality. The small size and limited functionality make the kernel more likely to be correct and worth formally verifying.

The concepts of microkernel and separation kernel have many similarities. The kernel is minimized in both concepts, including only a few functions to support a userspace system. The main difference is that a microkernel includes enough functionality to support general-purpose OSs, sacrificing some isolation, while a separation kernel can only support specialized systems.

seL4 is a formally verified L4 microkernel [25]. Liedtke [24] created the original L4 microkernel with a focus on performance as earlier microkernels were too slow for practice. seL4 is a high-security version of the L4 microkernel, focusing on both security and performance. seL4 is also configurable, allowing it to operate as a separation kernel [21] or RTOS. Heiser et al. [25] verified seL4 in Isabell/HOL theorem prover and implemented it in C, but they designed it in Haskell. Haskell was chosen as an intermediate language between Isabell/HOL and C, enabling quick prototyping and verification. Once they finalized the prototype, they translated it into to C. The C implementation was then formally verified on a source code level using so-called refinement proofs against a formal specification of the kernel execution.

Other kernels. *CertiKOS* is a secure hypervisor for cloud computing with a minimal kernel. The CertiKOS kernel handles only the delegation

[1] In a semiformal verification, we show that (a) the functional specifications of the system implements the security policy model, (b) the high-level design of the system implements the functional specifications, and (c) the low-level design of the system implements the high-level design. Note that only the functional specification and the security policy model must be formally defined, while the high- and low-level designs of the system are only required to be semiformal representations of the system.

of the CPU cores, RAM, hard drives, and network I/O to the guests. The guest handles the fine-grained management of these resources. The kernel treats the provider's management facility as a guest with special privileges to delegate and set access policies on the resources. The kernel enforces the policies and securely manages the actual resources.

2.4 Side-Channel Attacks

Both TEEs and security kernels are sensitive to side-channel attacks. A side-channel attack targets the implementation of a computer system, rather than the algorithm. Side-channel attacks often require technical information on both the hardware and the software. In this thesis, we mainly consider timing and cache attacks, but other attacks are also important to consider.

Timing attack. In a timing attack, an attacker measures deviations in computational time to infer what kinds of computations have occurred. These deviations can be used in, for example, inferring keystrokes [26], or passwords. We should consider timing attacks for both security kernels and TEEs, but this is rarely done.

Cache attack. In a cache attack, an attacker monitors the cache access made by the victim. In 2017, Lipp et al. [1] found cache vulnerabilities resulting in privileged information leakage on several processors, and in 2018, Van Bulck et al. [27] revealed that even Intel's SGX enclaves could leak information through a cache attack.

DMA attack. In a DMA attack, an attacker exploits the direct memory access (DMA) controller to circumvent the kernel and processor protection. Because the DMA has direct and unimpeded access to the physical memory address space, the attacker will have access to all memory-mapped devices such as the main memory.

Other attacks. Other side-channel attacks, such as power-monitoring, are also important to consider. An untrusted application that can monitor the power consumption can use this ability to infer what a supposedly secured application is computing. We can not, however, solve this problem *efficiently* with a kernel or TEE.

2.5 Secure Interrupts

There are a few problems when implementing interrupts securely:

(a) Secure and non-secure interrupts can occur in non-secure and secure domains, respectively, requiring security domain transitions [28].

(b) An interrupt is sensitive to timing attacks [26]. A program can measure time to observe timing, revealing that some interrupt has occurred, and perhaps even the type of interrupt.

(c) An attacker can exploit an interrupt to make denial-of-service attacks. The interrupt handling processes preempts other processes, so if an attacker can trigger an interrupt frequently enough, a system slows down.

Problem (a) can be solved [9, 28] , but problems (b) and (c) are either unsolvable or requires significant sacrifices. Problem (b) can be solved by disabling interrupts in general and instead schedule short time slices where interrupts are enabled and can be handled. However, this solution will incur significant latency on the interrupt handler and waste time, which is often unacceptable in safety-critical applications.

We can solve problem (c) using (b)'s solution, or by budgeting the interrupt handler's time slice. However, this solution will leak some timing information, but under normal conditions, it should never incur significant latency.

2.6 CompCert

Several studies have demonstrated bugs in compilers resulting in miscompilation [29, 30]. Miscompilations are bugs in compilers that silently generate incorrect executable code from correct source files. These bugs are at most annoying for everyday applications, but in safety-critical and mission-critical applications, it can be disastrous. Moreover, miscompilations invalidate formally verified software, static analysis, and model checkers.

CompCert is a formally verified C compiler that accepts almost all of the ISO C99, and ANSI C languages [31]. The formal verification of CompCert consists of proving the following semantic preservation theorem:

> For all source programs, S and compiler-generated code C, if the compiler, applied to the source S, produces the code C, without *reporting a compile-time error*, then the observable behavior of C improves on one of the allowed observable behaviors of S. [31]

The observable behaviors of a source program S are everything that can be observed externally by a user or the physical world, except for execution time and memory consumption. The externally observable behavior includes: program termination, I/O, and writes to volatile global variables. How the possible observable behaviors are defined is the purpose of formal semantics, a mathematically-defined relation between programs and behaviors.

The semantic preservation theorem tells us that the observable behavior of the compiler-generated code C is always one of the possible observable behaviors of the source code S. The generated code C can be an improvement of the source code S since the semantic preservation does not prohibit optimizations such as dead code elimination.

Leroy et al. [31] has proved the semantic preservation theorem for CompCert using the Coq proof assistant, letting S be a C-level abstract syntax tree (AST), and C be an assembly-level AST. Note that CompCert does not support the whole C99 standard, and can, therefore, report compile-time errors for parts of C99, but it will never produce a miscompilation.

On PowerPC and ARM, CompCert code runs at least twice as fast as GCC without optimizations, and approximately 10% slower than GCC with optimization level 1 [31]. Leroy et al. [31] claims that developers of safety- or mission-critical applications consider this performance level acceptable since they rarely use GCC with optimizations.

Chapter 3

Methods

In this chapter, we present the design goals of OpenMZ, our open-source version of HexFive's MultiZone Security kernel, and how we evaluate it.

3.1 Design

We build and design OpenMZ targeting RISC-V with U- and M-mode privilege levels, 8 PMP registers, and no N-extension. The target applications of OpenMZ are secure embedded devices and secure coprocessors. We implement all of the MultiZone API systems calls, as described in Subsection 2.2.4. We implement and evaluate OpenMZ with the following design goals:

Verifiability. To formally verify the kernel, both the low- and high-level design of the kernel must be understandable and straightforward. For this purpose, we implement OpenMZ mostly in C, using assembly only where it is necessary. This allows us to understand it easier than the MultiZone Security kernel, which is implemented in pure assembly. Moreover, if we can compile our kernel with CompCert, we can verify the kernel at C-level, making formal verification easier. Furthermore, to evaluate our C implementation's simplicity, we present the kernel's high-level design and measure the number of lines of code.

Small memory footprint. We design OpenMZ for embedded applications and secure coprocessors lacking S-mode. These applications tend to have a limited memory or processing power, requiring the kernel to have a small memory footprint. Therefore, to evaluate this goal, we measure the binary size of the kernel. In this measurement, we limit

ourselves to two zones, since the number of zones will only affect the size of the data section.

High-security. We design the kernel to effectively isolate zones and support secure interrupts using M-mode features such as the PMP mechanism, and the delegation of interrupts and exceptions to lower privilege levels. So to evaluate this goal, we define a set of attacker models (Section 3.2) and analyze the security of the kernel with respect to these attackers.

Real-time properties. We design the kernel such that it either (a) guarantees that some minimum amount of execution time is available to each zone in a set period or (b) brings an interrupt handler into context immediately upon an interrupt. We then analyze the kernel to find out if and when these properties can be fulfilled.

High-performance. We design the kernel with performance in mind by optimizing the kernel's most common operation; that is, the context switch. So to evaluate the fulfillment of this goal, we design benchmarks measuring the cost of context switches (Section 3.4).

3.2 Attacker Models

We define three classes of attackers who aim to compromise the security of OpenMZ. In these attacker models, we assume that we set up the zones so that their PMP regions do not overlap.

Data-based attack. A zone tries to read or modify the data of other zones or the kernel in an unintended manner.

Cache attack. Zones are trying to extract information from other zones or the kernel using cache attacks.

Timing attack. Zones are trying to extract information from other zones or the kernel using timing attacks.

3.3 Size Benchmark

We measure both the code and binary size of OpenMZ. We count the number of lines of C, C header, and assembly code using `cloc` version 1.82 [32]. We compile OpenMZ with CompCert version 3.7 [33], and with the RISC-V GCC version 9.2.0 [34]. We measure the binary size using

RISC-V `size` tool version 9.2.0 [34]. We compile OpenMZ targeting the HiFive1 rev. B development board [35], which has the RV32IMAC ISA[1].

3.4 Performance Benchmark

Hardware. We measure OpenMZ's performance on the HiFive1 rev. B development board [35]. The HiFive1 card has the FE310 system-on-chip which includes: SiFive's E31 core, implementing RV32IMAC with user and machine mode; 16 KiB [2] of 2-way L1 instruction cache; 16 KiB of SRAM scratchpad; flexible clock generator capable of generating a clock frequency of up to 320 MHz; and peripherals such as UART and QSPI.

The E31 execution unit is a single-issue, in-order, five-stage pipeline. The pipeline has a peak execution rate of one instruction per clock cycle and is entirely bypassed, so most instructions have a one-cycle latency. The exceptions are load instructions, CSR reads, multiplication, and division.

By default, FE310 stores its instruction in an external QSPI flash memory device; thus, cache misses have a significant impact on the system performance. To alleviate the impact of cache misses, we can configure HiFive1's instruction cache to store 8KB of instructions that would otherwise be in the QSPI flash memory device. Programs stored in the cache are then unaffected by cache misses, but this is at the cost of cache memory for the rest of the system. Another option is to store the instructions in the SRAM scratchpad as it allows executable code. The scratchpad is, however, very small and optimized for data.

Metrics. We measure OpenMZ's performance by counting the number of additions the zones can execute during 10 seconds, and we call this *work* (W). We count the number of additions using the following RISC-V assembly function:

```
adder:
        addi  a0,a0,1
        addi  a0,a0,1
        ...
        addi  a0,a0,1
```

[1] RV32IMAC is the base RV32I ISA with integer multiplication and division (M), atomic instructions (A), and compressed instructions (C)

[2] Testing shows that the actual cache size is closer to 15,968 bytes.

```
j adder
```

which repeatedly adds 1 to register `a0` in a loop until interrupted. Before this function runs, a 10-second timeout is set using OpenMZ's timer interrupt for zones. When the timeout has been triggered, we have the work in register `a0`. Because the `addi` and the `j` instructions are both one-cycle instructions, the work is an approximation of the effective number of cycles spent in the zone. The exact length of the `adder` function is specified under "Cache issue."

We let our *baseline work*, W^b, be the amount of work one zone can perform by itself, i.e., when we have no context switches. We define W_t to be *time slice work*, or, the sum of all work performed by the zones when we have a round-robin scheduler with a t ms time slice.

With W_t and W^b, we can calculate OpenMZ's *kernel overhead* O_t by

$$O_t = 1 - \frac{W_t}{W_b},$$

which is the fraction of time lost on context switches and cache misses. An upper-bound to the total time spent on context switching during one second is thus approximately O_t seconds. We approximate the *context switch time* C_t by

$$C_t = \frac{O_t}{F_t},$$

where $F_t = \frac{1000}{t}$ is context switches per second.

Cache issue. We get the worst-case performance of a context switch by measuring it when the kernel code is not in the instruction cache. The kernel flushes the cache if it can, to prevent cache attacks; we expect, therefore, that the average-case performance is the worst-case performance. The FE310 processor does, however, not have the ability to flush the cache directly.

We circumvent FE310's cache-flushing issue by creating a 16 KiB (or exactly 15,968 bytes) long version of the `adder` function called `adder_evict`. The `adder_evict` function has then 7982 add instructions[3] and one jump instruction[4]. The `adder_evict` function is then long enough to evict the entire instruction cache, allowing us to measure the worst-case performance if the time slice is long enough.

[3] An `addi` instruction with the same source and destination register is 2 byte wide when compiling with the C-extension.

[4] If the offset is within ±2 KiB, the jump instruction is 2-byte wide. If the offset is within ±1 MiB, the jump instruction is 4-bytes wide.

When benchmarking, we set the FE310 core clock to 192 MHz. In FE310, the `addi` is a one-cycle instruction, thus it takes at least 41 μs to complete one pass of the `adder_evict` function and evict the cache. This lower bound does not account for instruction cache misses. A hit on the cache has an access latency of one cycle, but the latency on misses is unknown. We expect that the access latency on misses to be in μs because the instructions are in the QSPI flash. Since we have an instruction cache with 32-byte lines, we expect up to 500 cache misses on one pass of the `adder_evict` function, after which we have only cache hits. So when the cache is clear, we expect that one pass of the `adder_evict` code requires a few ms. Because the `adder_evict` function is heavily affected by cache misses, the overhead, O_t, does not only reflect the time lost on context switches but also on the zones' cache misses.

For measuring the context switch time when we do not have cache-flushes, we create the `adder_no_evict` function, which consists of 15 add instructions[5] and one jump instruction[6]. The `adder_no_evict` is then 32 bytes long and occupies one HiFive1 cache line.

Setup. We calculate the work (W_t), overhead (O_t), and context switch time (C_t), for $t = 1$, 5, 10, 25, 50, 75, 100, 150, 200, 250, 500, and 1000 using two zones. We concentrate the benchmarking for t between 10 and 100 ms, as this is a common time slice range. We benchmark on $t < 10$ since it demonstrates the impact of cache misses on the `adder` functions when the time slice is short. For $t > 100$ ms, we see how the kernel overhead behaves asymptotically.

We benchmark the kernel with the two different `adder` versions, the `adder_evict` function, and `adder_no_evict` described previously. Moreover, we compile the kernel and the benchmarking code using the RISC-V GCC compiler version 9.2.0 [34] with `-O2` optimization.

[5]See footnote 3.
[6]See footnote 4

Chapter 4

Results

4.1 Implementation

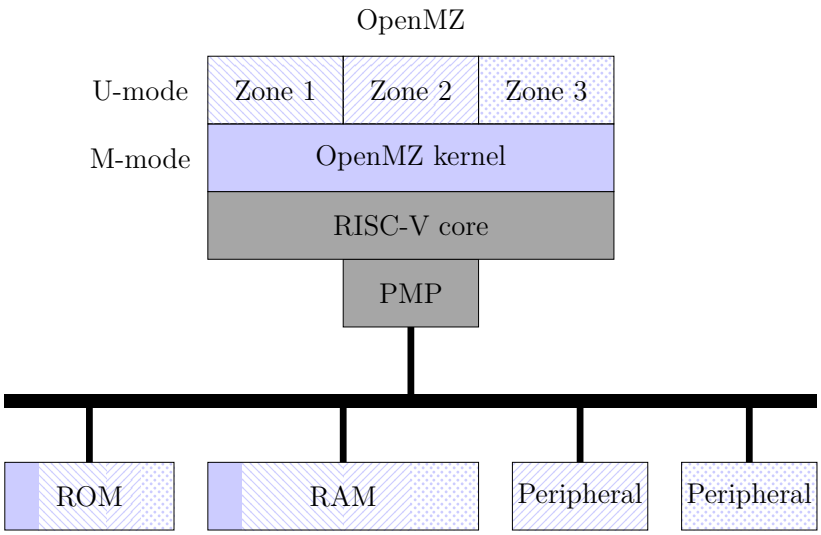


Figure 4.1: The OpenMZ kernel partitions a system into several isolated zones running in U-mode and provides services to them. Each zone has up to eight memory-mapped regions, the first two meant for instructions and data, and the rest for peripherals.

Kernel responsibilities. Like MultiZone Security, OpenMZ partitions a system into several zones running in U-mode, and manages them with an M-mode kernel (see Figure 4.1).

The M-mode kernel is the core of OpenMZ, and it has the following responsibilities:

Enforce isolation between zones. Each zone is associated with a set of PMP configurations. Before the kernel brings a zone into context, the kernel flushes the cache and loads the zone's PMP configuration to the PMP registers. Since a zone runs in U-mode, its PMP configuration defines the zone's resources.

Schedule zones. OpenMZ uses a round-robin or cooperative scheduler. When it uses the round-robin scheduler, it uses the machine timer interrupt to preempt the zones. If the kernel uses the cooperative scheduler, the zones use the yield system call to hand over control.

Delegate exceptions. For each zone, the kernel stores an array of pointers to the zone's exception handlers (Figure 4.2, the `excpt_handlers` array in `zone_t` struct). When a zone triggers an exception, the kernel saves the zone's trap information, disables the zone's interrupts, and sets the zone's `pc` to the corresponding exception handler.

A zone's trap information consists of the following: `uepc`, the `pc` at the exception point; `ucause`, corresponding to `mcause`; `utval`, corresponding to `mtval`. The current OpenMZ implementation does not have a method for the zones to read or write to these trap variables, but it will be implemented in the future.

Delegate interrupts. The kernel contains an array mapping interrupts to zones (Figure 4.2, the `intrp_handlers` array in `kernel_t`) where element i is a mapping for interrupt i. Each mapping consists of a reference to the zone and a pointer to the interrupt handler.

When an interrupt is triggered, the kernel does the following:

- brings the associated zone into context,
- saves the trap information,
- disables the zone's interrupts,
- sets the zone's `pc` to that of the interrupt handler,
- disables the interrupts globally (except the timer interrupt when we use the round-robin scheduler), and
- sets the "interrupt mode" flag (Figure 4.2, `intrp_mode` in the `kernel_t` struct).

Interrupts are enabled globally once the kernel exits the "interrupt mode" which happens when the zone is preempted, yields, or calls the `uret` instruction.

We can delegate almost all CLIs to a zone; it is only the machine timer interrupt that we can not delegate since the kernel uses the timer interrupt.

Emulate `uret`. To allow zones to restore their context after handling delegated exceptions and interrupts, the kernel emulates the `uret` instruction. A zone that executes a `uret` instruction triggers an illegal instruction exception since we have no N-extension, invoking the kernel.

On an illegal instruction exception, the kernel checks which instruction that caused the illegal exception. If the instruction is not a `uret`, the kernel delegates the exception to the zone. If the instruction is `uret`, the kernel emulates the `uret` instruction, and then, if the kernel is in the "interrupt mode", it restores the interrupted zone context; otherwise, it restores the current zone context.

Service system calls. If a zone executes an `ecall` instruction, the kernel's system call handler is invoked. The system call handler advances the program counter, and if the system call number is valid, executes the corresponding system call. OpenMZ implements the system calls of the MultiZone API described in appendix B. The kernel implements all system calls of the MultiZone API as specified except `ECALL_WFI`, which we legally implement as `ECALL_YIELD`.

Inter-zone communication. In the kernel, we have a matrix of inboxes (Figure 4.2, the `inboxes` matrix in the `kernel_t` struct) where inbox (i, j) contains at most one message from zone j to zone i. Each inbox has a `full` flag and a 128-bit message.

When zone i sends a message to zone j, it places j and the message in its registers, `a0-a4`, and then invokes the `ECALL_SEND` system call. The kernel then checks if inbox (j, i) is full. If the inbox is empty, the kernel copies the message from zone i's registers to the inbox, sets the `full` flag, and returns 1. If the inbox is full, the kernel returns 0.

The kernel never notifies the zones of new messages, so they must poll the inboxes using the `ECALL_RECV` system call to receive new messages. When zone i polls for a message from zone j, it uses the `ECALL_RECV` system call. If inbox (i, j) is full, the kernel copies the message to the receivers registers, unset the `full` flag, and returns 1. If the inbox is empty, the kernel returns 0.

Emulate timer interrupts. Each zone can have an asynchronous timer interrupt. The kernel uses the machine timer interrupt to set the zone's timer interrupt, when it expires before the end of the zone's time slice, meaning that the zone's timer interrupt is asynchronous. Moreover, the zone's timer interrupt is only active when the zone is in context to protect the other zones.

Kernel data. The kernel stores all global data in one struct for ease of management. In Figure 4.2, we see the data structs used in the kernel. It is the `kernel_t` struct that contains all of the data.

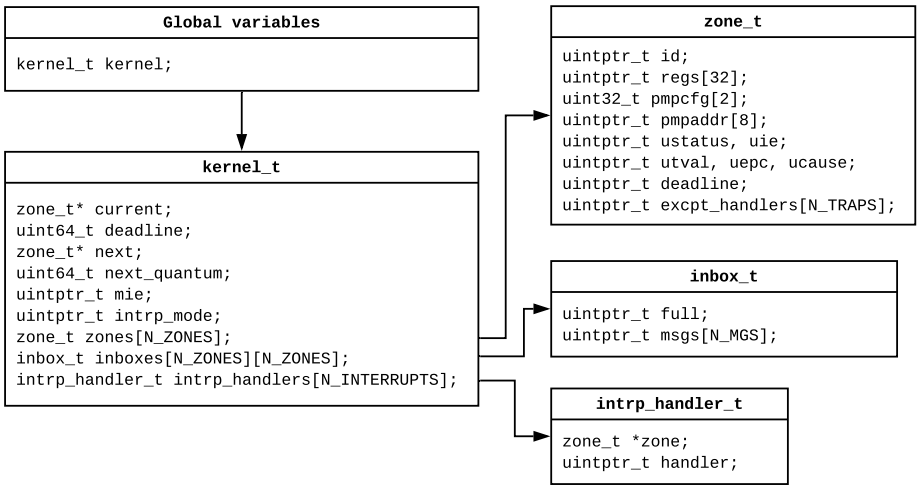


Figure 4.2: OpenMZ's structs. The only global variable in the kernel is an instance of the `kernel_t` struct which contains all the information the kernel needs regarding the zones, scheduling, and messages.

Kernel functions and control flow. We implemented the core of the kernel in C and the trap entry/exit functions in assembly code. In Figure 4.3, we see an abstract view of the kernel's control flow. Most of the assembly code is in the assembly functions prefixed with `asm`, but the kernel does have a few lines of inline assembly code in the C part of the kernel for reading and writing to CSR registers.

The kernel has two entry points and one exit point. The first entry point is the kernel main function, `kmain`, which initializes the CSRs, zones, scheduler, and hardware. The second entry point is the trap vector, `asm_trap_entry`, which saves the zone's context and then calls the trap

handler. The kernel's exit function is `restore_context`, it flushes the cache, sets the PMP configuration, and restores the zone's context.

The core of the kernel is in between the trap handler, `handle_trap`, and exit function. The trap handler is responsible for calling the correct handler, then calling the scheduler, and finally, the kernel exit function. The handlers are all designed to be tail-callable, minimizing function overhead. The scheduling function, `sched`, handle the context switching (e.g., round-robin) and interrupts.

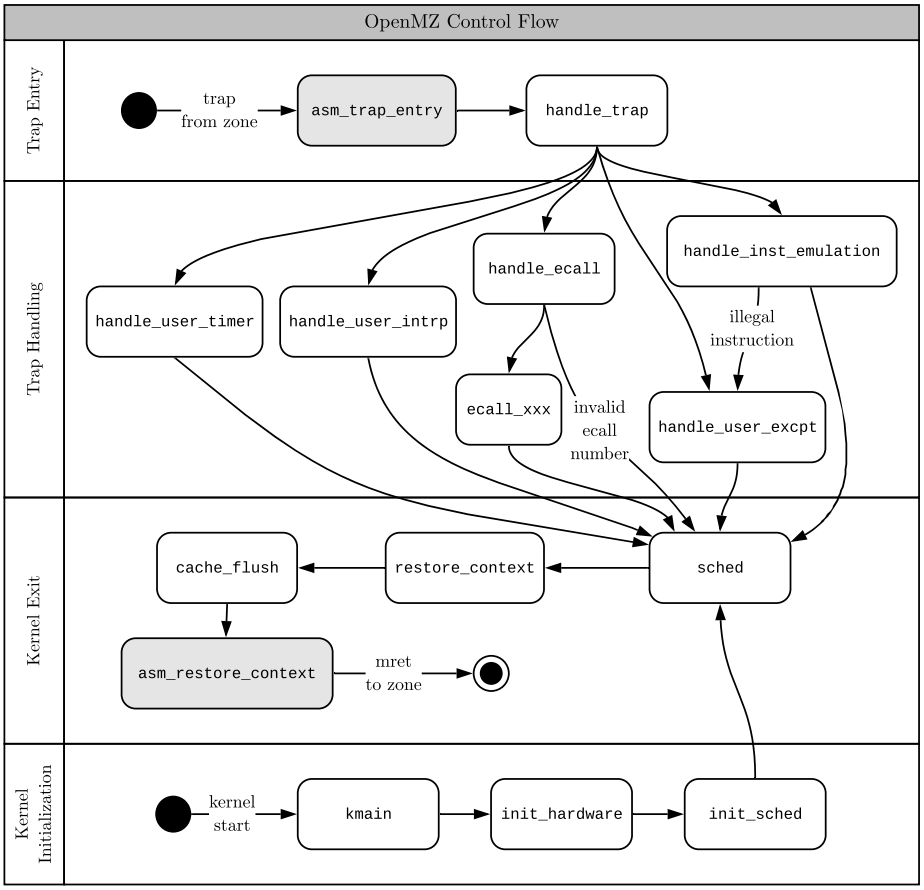


Figure 4.3: An overview of OpenMZ's control flow. Functions were designed so the control flow graph is directed and acyclic. All nodes (except ecall_xxx) correspond to one major function. Assembly function are marked with grey and prefixed with `asm_`.

Configuration. We configure the zones by altering the initial kernel struct, i.e., by hard-coding. The zone struct's PMP fields are configured exactly like RISC-V's PMP configurations, the initial `pc` is stored in `regs[0]`, and the interrupt mapping is stored in the `intrp_handlers` array. To make the configuration easier for the developer, we use C-macros.

4.2 Size Benchmark Results

Code size. In table 4.1, we see the number of lines of C code, C header code, and assembly code used in OpenMZ, this is excluding blank lines and comments.

Type	Files	Lines
C	4	370
C header	8	232
Assembly	2	156
Total	14	758

Table 4.1: Number of files and lines of code. Excludes board support kits.

Binary size. In table 4.2, we see the sizes of the OpenMZ binary using different compilers and optimization level. As expected, the size of the data and bss sections remain the same, but the size of the text section varies. The largest binary is that of `gcc -O0` at 4 KiB, followed by CompCert at 3.4 KiB. The binary sizes of gcc with optimizations are all around 2 KiB. The bss section consists of the stack, which is by default set at 512 bytes.

	`gcc -O0`	`gcc -O1`	`gcc -O2`	`gcc -O3`	`ccomp`
.text	4032	2240	2080	2128	3472
.data	888	888	888	888	888
.bss	512	512	512	512	512
Total	5944	4152	3992	4040	5384

Table 4.2: OpenMZ binary size in bytes when targeting the HiFive1 rev. B development board [35] that has the RV32IMAC ISA.

4.3 Performance Benchmark

Cache-eviction. In table 4.3, we have the performance metrics on the cache-evicting version of our performance benchmark. The results show that the cost of the context switch is around 1,650,000 cycles or 8.55 ms, resulting in a kernel overhead that falls from 86% to 9% when going from 10 ms to 100 ms time slices. Note that the cost of the context switch also includes the cost of the cache misses. For time slices of less than 10 ms, it is not long enough to allow the `adder_evict` function to fully evict cache, thus C_t is less than 5 ms due to fewer cache misses.

The fraction of add instructions in `adder_evict` is $\frac{7982}{7983}$, thus the maximum theoretical baseline is 1,919,759,488, which is very close to the measured baseline.

t (ms)	W_t	O_t (%)	C_t (ms)	Cycles
1	15,532,718	99.19	0.99	190,446
5	15,871,370	99.17	4.96	952,061
10	262,326,373	86.33	8.63	1,657,570
25	1,255,812,197	34.57	8.64	1,659,234
50	1,589,392,250	17.19	8.59	1,649,911
75	1,698,186,795	11.52	8.64	1,658,586
100	1,755,118,243	8.55	8.55	1,641,909
150	1,808,903,588	5.75	8.62	1,655,766
200	1,837,217,422	4.27	8.55	1,641,188
250	1,854,192,399	3.39	8.47	1,626,943
500	1,886,821,161	1.69	8.45	1,621,806
1000	1,902,948,762	0.85	8.49	1,630,218
Baseline	1,919,323,331			

Table 4.3: Performance metrics on the cache-eviction case. We get cycles per context switch by $C_t \times$ clock frequency. The kernel and the zones were compiled with `-O2` optimization with GCC. The benchmark was performed on the HiFive1 rev. B development board [35].

No-Cache-Eviction. In table 4.4, we have the performance metrics of the no-cache-eviction case. The results show that the context switching time increases with the time slice length. At 1 ms time slice, we have a 1.57 μs context switch while the 1000 ms time slice has a 5.36 μs context switch, so a context switch uses 300-1000 cycles.

The fraction of add instructions in `adder_no_evict` is $\frac{15}{16}$, thus the maximum theoretical baseline is 1,800,000,000 which is very close to the measured baseline.

t (ms)	W_t	O_t (%)	C_t (μs)	Cycles
1	1,797,111,094	0.16	1.57	302
5	1,799,379,237	0.03	1.55	298
10	1,799,660,546	0.02	1.54	296
25	1,799,821,572	0.01	1.61	310
50	1,799,876,562	0.00	1.70	327
75	1,799,893,187	0.00	1.86	357
100	1,799,907,236	0.00	1.70	326
150	1,799,913,699	0.00	2.01	386
200	1,799,916,161	0.00	2.41	463
250	1,799,919,090	0.00	2.60	500
500	1,799,925,571	0.00	3.41	654
1000	1,799,928,194	0.00	5.36	1,029
Baseline	1,799,937,841			

Table 4.4: Performance metrics on the no-cache-eviction case. We get cycles per context switch by $C_t \times$ clock frequency. The kernel and the zones were compiled with `-O2` optimization with GCC. The benchmark was performed on the HiFive1 rev. B development board [35].

4.4 Real-Time Properties

In Section 3.1, we presented two real-time properties we want from the kernel: (a) guarantee that a minimum amount of execution time is available to each zone in a set period, or (b) bring an interrupt handler into context immediately upon an interrupt. The kernel can not fulfill these two properties simultaneously as they interfere with each other. OpenMZ can, however, fulfill at least one of them at a time.

Guaranteed execution time. OpenMZ guarantees (a) when there are no interrupts for zones. Let N be the number of zones. The round-robin scheduler assigns time slices in equal proportion to each zone in a cyclic fashion, let this time slice be t. The scheduler is simple and starvation-free, guaranteeing that the kernel brings a zone into context within a set amount of time. OpenMZ uses the RISC-V's machine timer

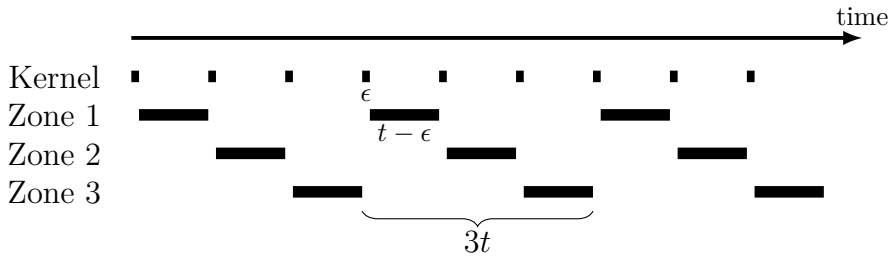


Figure 4.4: Three zones scheduled using round-robin with a time slice of t, and no zone-interrupts. During every $3t$ period, a zone is guaranteed to receive at least $(t - \epsilon)$ runtime, where ϵ is the upper-bound of the context switch time and the system call time combined.

interrupt to preempt the zones, ensuring that zones use only their time quota.

With the round-robin scheduler and no zone-interrupts, OpenMZ guarantees (a) because each zone receives at least $(t - \epsilon)$ execution time in a period of length Nt where ϵ is the upper-bound of the context switch time and a system call time combined (see Figure 4.4). We can approximate ϵ using the performance measurement method from Section 3.4. Note that if a zone yields, it has still *received* $(t - \epsilon)$ execution time. If OpenMZ has interrupts mapped to a zone, (a) can not be fulfilled since an interrupt steals execution time from the zones.

Immediate interrupt handling. OpenMZ guarantees (b) when there is only one interrupt assigned to a zone, and the kernel is not in context (see Figure 4.5). When an interrupt assigned to a zone Z triggers, the following happens:

1) The kernel suspends the current zone, setting it to be the next zone to run, and with its remaining time slice.
2) Zone Z is set as the current zone, and scheduled time slice t.
3) The kernel saves the zone's trap information to the zone's struct, clears the zone's global interrupt bit, and sets the zone's `pc` register to the interrupt handler.
4) The kernel enables only the timer interrupt and brings zone Z into context.

The kernel is not preemptible, so if the kernel is in context when the interrupt triggers, the kernel brings zone Z into context only after the kernel exits.

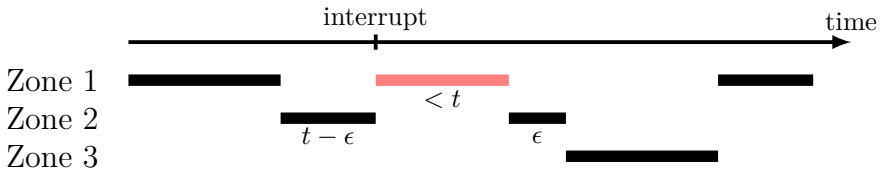


Figure 4.5: Three zones with zone 1 having an interrupt and zone 2 being interrupted. Zone 1's interrupt handler runs at most t seconds, zone 2 is then brought back with its remaining time slice.

4.5 Security Analysis

In this section, we first analyze data-based attacks and then timing vulnerabilities. Attacks requiring physical access are out-of-scope.

4.5.1 Data-Based Attacks

We assume that all zones have a valid PMP configuration; that is, the zones can not read or write to the kernel's resources. If the PMP configurations are not valid, we have no security guarantees. We also assume that the RISC-V implementation the kernel is running on can flush the cache.

Protection of the kernel. If the zones' PMP configurations are valid, the PMP mechanism prevents the zones from directly accessing the kernel's data. When the kernel brings a zone into context, it loads the zone's PMP configurations and runs it in U-mode. Therefore, the PMP unit checks the zones' memory access, preventing it from accessing resources outside of its domain.

Zones can also attempt to infer information from the kernel using cache attacks. If the RISC-V implementation can flush the cache, the kernel flushes the cache before bringing a zone into context, and, therefore, the kernel protects itself from cache attacks.

Zones can attempt to extract information or compromise the kernel through system calls. OpenMZ system calls do not compromise the kernel because of the following reasons: the system calls read arguments from the zone's registers; the kernel checks all array accesses used in system calls; system calls can only access a narrowly defined set of data; the system calls contain no loops (termination); most system calls are straightforward.

Moreover, the kernel does not use any buffers or dynamically allocated memory, making overflow attacks impossible.

Isolation of zones. In OpenMZ, the zones are allowed to share memory-mapped resources, but we do not recommend this. Assuming we have configured the zones' PMP such that no zones share any resource, we can guarantee the memory isolation of the zones. As mentioned earlier, the kernel guarantees that the zone runs in U-mode with its PMP configuration applied. A zone can, therefore, not directly access the memory-mapped resources of another zone. Moreover, the kernel protects the PMP configurations throughout the execution, so a zone can not modify its PMP configuration to get access to more resources.

Zones can also attempt to infer information from other zones using cache attacks, but the kernel prevents this using cache-flushes.

4.5.2 Timing Vulnerabilities

Due to the MultiZone API, OpenMZ has multiple timing-related vulnerabilities. Specifically, the interrupts and the `ECALL_YIELD/WFI` system calls are the parts of the MultiZone API that introduce these vulnerabilities.

Vulnerabilities by interrupts. In Section 2.5, we presented a few problems when implementing interrupts securely, and these problems apply to OpenMZ's interrupts as well.

When a zone-interrupt is triggered, the kernel brings the assigned zone into context, preempting the current zone. When preempting the current zone, the kernel saves its remaining time slice, causing a shift in the scheduling time of all zones, as shown in Figure 4.6.

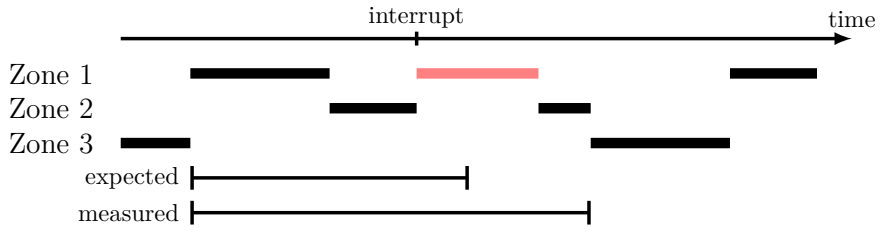


Figure 4.6: Zone 3 is measuring the time between its contexts. The measured time is longer than the expected time, indicating that an interrupt has occurred.

Vulnerabilities from the yield or WFI system calls. Contrary to the interrupt vulnerability, a yield or wfi causes a hastening of the

scheduler. This hastening can also be detected and measured by other zones, as seen in Figure 4.7.

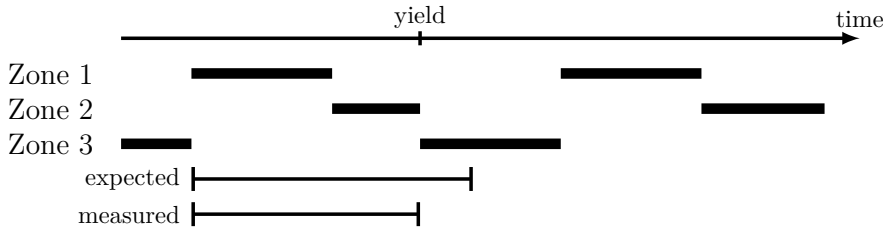


Figure 4.7: Zone 3 is measuring the time between its contexts. The measured time is shorter than the expected time, indicating that some zone did a yield or wfi call.

Vulnerabilities from system call handling. If a zone does a system call at the end of its time slice, the time slice might end during the system call handling. OpenMZ does not protect against this scenario, so these system calls can result in a slight delay of the context switch, which other zones may detect.

Chapter 5

Discussion

Achievement of design goals. In Section 3.1, we set the following five design goals for the kernel: verifiability, small memory footprint, high-security, real-time properties, and high-performance. Here we discuss if, or to what degree, we have achieved these goals.

Verifiability. OpenMZ's trusted code base is tiny, a total of 758 lines of C and assembly code, and it is a separation kernel, making it suitable for formal verification. The seL4 kernel is formally verified while having 8,700 lines of C code and 600 lines of assembly code [25], significantly more than OpenMZ. Code size is, therefore, not a problem for OpenMZ when it comes to verification. We can also compile OpenMZ CompCert so that we can formally verify it at C-level. Moreover, OpenMZ is a separation kernel, so new device drivers and system services do not affect the integrity of the kernel as they are implemented in the zones. The formal verification of OpenMZ is, therefore, worthwhile as these additions do not invalidate it.

Small memory footprint. The kernel has a small memory footprint, requiring only 2-4 KiB of flash memory, and 1.5 KiB of RAM. OpenMZ's size is comparable to that of HexFive's MultiZone Security (MZS) that requires at most 4 KiB of flash and 1 KiB of RAM [36]. MZS's size does, however, include a secure bootloader, which OpenMZ does not. If we add a secure bootloader to OpenMZ, the size could be roughly 8 KiB, larger than MZS, which we expect since the MZS kernel is written in assembly.

High-security. OpenMZ isolates the zones' memory using the PMP mechanism, prevents cache attacks with cache-flushes, but does not protect

against timing attacks. The most significant timing attacks comes from the MultiZone API's zone-interrupts, and `ECALL_YIELD/WFI` system calls, which leaks information. Moreover, zone-interrupts makes the system vulnerable to denial-of-service attacks. This is because the zone-interrupts preempts the zones, so frequent interrupts will starve them. To secure OpenMZ, we would have to disable these features, changing the kernel and its API. Removing these features would, however, hurt the real-time properties of the kernel, so we need a better scheduler to improve upon these properties.

We also have a timing vulnerability from the kernel's system call handler, which may slightly delay the context switch. This vulnerability is, however, probably harder to exploit due to variances in context switching time, and the rarity of the vulnerable event. We can fix this vulnerability by postponing the system call handling when the remaining time slice is too short for handling the system call. The kernel should instead wait until the time slice expires and then execute a context switch. The zone's system call will then be handled during its next context.

Lastly, we should note that the PMP mechanism does not protect against DMA attacks since the DMA circumvents the PMP mechanism. So, OpenMZ is powerless against a zone with access to the DMA controller, and it is for this reason that we did not include the DMA attack in our attacker model. But if we have a DMA controller in the system, we should formally verify all zones with access to the DMA controller to maintain the validity of the kernel verification. This is the approach taken by seL4 [37].

Real-time properties. Our goal was to design the kernel so that it fulfills either (a) guarantees that some minimum amount of execution time is available to each zone in a set period, or (b) brings an interrupt handler into context immediately upon an interrupt. We have shown that the OpenMZ kernel fulfills (a) if we do not have immediate interrupt handling, and (b) if we only have one zone with immediate interrupt handling. We do not know the minimum execution time for (a) as this requires us to know the worst-case execution time of a context switch.

High-performance. The OpenMZ kernel's context switch costs, on average, 500 cycles on the E31 processor when we have no cache misses. The context switch cost increases with time slice length, the reason for this is unknown, but we suspect that it might be related to branch mispredictions. When the time slice increases in length, the

context switch is executed less frequently, so the branch predictor is less optimized. Another suspect is an initial cache-miss caused by the benchmark setup code, the first two context switches would evict this code, so the subsequent context switches will be free of cache-misses. Benchmarks with fewer context switches, i.e., longer time slices, would have, on average, a longer context switch due to these initial cache-misses.

A 500 cycle context switch is reasonably efficient, considering that seL4's IPC call uses, on average, 500 cycles on a HiFive Unleashed development board [1] [39]. The seL4 IPC call combines message passing with a user-initiated context switch [40], so the comparison is fair. However, a comparison with MZS is more interesting since it is an implementation of the MultiZone API, but this would require us to benchmark MZS.

The cost of the context switch when including cache misses is 8.55 ms or 1.6 million cycles, which is unacceptable. However, this high cost is due to the E31 processor that fetches instructions from a flash memory device on cache misses. So, from this result, we can conclude that the HiFive1 rev. B memory system is slow. Therefore, we are interested in benchmarks with more efficient processors so that we can better understand the impact of cache misses on the kernel.

Relevance. OpenMZ's main alternatives are MultiZone Security, Keystone, seL4, ARM TrustZone, and Intel SGX (and the AMD equivalence). ARM TrustZone and Intel SGX are for proprietary hardware, so they are not an option when we need verified hardware. Keystone and seL4 require processors with supervisor mode, something which OpenMZ and MultiZone Security can not handle. OpenMZ and MultiZone Security can, however, be used for secure coprocessors using a Linux companion (future work for OpenMZ), and be used in secure embedded systems (M- and U-mode only systems). OpenMZ's true competitor is, thus, MultiZone Security. MultiZone Security is written in assembly, meaning that it will probably be faster and smaller. MultiZone is, however, proprietary, meaning that OpenMZ can be the foundation of a fully-verified secure embedded system or secure coprocessor, while MultiZone Security can not. Moreover, the open-source nature of OpenMZ makes it suitable for future research and allows users to customize the kernel.

[1] HiFive Unleashed [38] has four RV64IMAFDC cores supporting the Unix-style operating system and one RV64IMAC monitor core. These cores have a very different feature set compared to HiFive1's E31 core, but their instruction execution rate per clock cycle is nearly identical.

Chapter 6

Conclusion

We implemented, benchmarked, and analyzed OpenMZ, an open-source version of the MultiZone API. Our target applications are secure coprocessors and secure embedded devices. Our design goals for OpenMZ are, therefore, verifiability, small memory footprint, high-security, real-time properties, and high-performance. Simplicity is essential for our long-term goal, the formal verification of OpenMZ. The other design goals are the essential properties of the kernel's target applications.

Our problem statement is "How well does OpenMZ fulfill the design goals?", and here is our answer.

Verifiability. The OpenMZ kernel is simple, having less than 800 lines of C and assembly code in total, making it suitable for formal verification. This size is far smaller than the formally verified seL4 kernel that has over 9000 lines of code.

Small memory footprint. The kernel has a small memory footprint of around 3-5 KiB, making it suitable for our target applications.

High-security. The kernel is isolating the zones' memory, but it has some significant timing-related vulnerabilities due to the MultiZone API. To fix these vulnerabilities, we must fundamentally change the API and the scheduler.

Real-time properties. The kernel is suitable for some real-time applications depending on the configuration and the features used. More specifically, the kernel guarantees (a) that some minimum amount of execution time is available to each zone in a set period when there are no zone-interrupts, or (b) brings an interrupt handler into context immediately upon an interrupt when we only have one zone-interrupt.

High-performance. We measured the performance of OpenMZ on SiFive's HiFive1 rev. B development board. OpenMZ context switch costs around 500 cycles if there are no cache misses. However, due to the development board's high cache-miss penalty, OpenMZ's context switches can cost up 1.6 million cycles, making it very inefficient.

Future work. It is important to note that at this stage, OpenMZ is not formally verified and will probably be modified for optimizations, features improvements, and bug fixes. Therefore, verification of the kernel is not the immediate goal, but it is the final goal. Here is a list of candidate features for OpenMZ.

Other schedulers. As mentioned in "Security", OpenMZ needs a better scheduler to fix the timing-related vulnerabilities. Here are some potential schedulers:

- Flexible Round-Robin. We can schedule a zone multiple times during one period with different time slice lengths. The kernel enables a zone's interrupts only when the zone is in context. We do not have the yield or wfi system calls.

  Example: The scheduling order $\{Z_1, Z_3, Z_2, Z_3\}$ where Z_1 and Z_2 are normal processes with 10 ms time slice, and Z_3 is an interrupt handler with a 5 ms time slice. The interrupt handler's maximum latency is then roughly 10 ms.
- Round-Robin with budgeted interrupt time. We use the current OpenMZ scheduler, but in every round-robin period, a zone gets a set amount of time to spend on interrupt handling. This budgeting stops DoS attacks and guarantees zones a set amount of execution time in a set period. This scheduler does, however, not fix the timing vulnerabilities.
- Round-Robin with smarter yield. When a zone yields, it gives its execution slot to another zone. Suppose we have three zones Z_1, Z_2, and Z_3 with the scheduling order $\{Z_1, Z_2, Z_3\}$. If zone Z_1 yields to zone Z_3, Z_1's remaining execution slot is given to Z_3, and Z_3 is immediately brought into context. Once the Z_1 slot has run out, Z_2 is brought into context, followed by Z_3 again.

  This scheduler removes the timing-related vulnerabilities we get from the yield system call, but it does not fix the issues from the interrupt delegation.

Flexible CSR API. We can make the API for accessing machine CSRs more flexible. The current API has one system call for every CSR we want to access, e.g., `ECALL_CSRR_MSTATUS()` for reading the `mstatus` register. We replace these system calls with one system call that takes a CSR code as an argument, e.g., `ECALL_CSRR(MSTATUS)`. Then, we can easily add or remove CSRs readable by the zones. Examples of future CSRs are the U-mode trap setup and trap handling CSRs which brings us to the next point.

Trap handling compatible with the N-extension. We can make the zones compatible with the N-extension with some extensive changes to the trap handling API. The current API forces the kernel to store a trap table and makes the kernel incompatible with the N-extension. If we want the kernel to be compatible with the N-extension, the kernel has to emulate the N-extension when the hardware does not implement it. To make the kernel compatible, we have to

- replace the arrays of trap handlers in the kernel with U-mode trap setup and trap handling registers,
- remove the `ECALL_TRP_VECT` and `ECALL_IRQ_VECT` system calls,
- make the U-mode registers read- and writeable using system calls, and
- emulate CSR instructions for reading and writing to the new U-mode registers.

CSR instruction emulation. The OpenMZ kernel does not emulate CSR reads of M-mode registers, although the HexFive's MultiZone Security emulates them. We left this feature out of OpenMZ due to time constraints and because it is not officially a part of the MultiZone API. The emulation of CSR-reads is, however, a desired feature.

Delegation of PLIs. The OpenMZ kernel delegates the interrupts based on the exception code, and since all PLIs share the same exception code, we can only map them to the same zone. This delegation method can be undesirable in situations where we want to assign the PLIs to different zones. An API for the fine-grained delegation of PLIs can, therefore, be interesting. Alternatively, an API that allows a zone to delegate the PLIs to other zones and notify them using an inter-zone interrupt scheme can also be interesting.

MultiZone Security benchmark. We compared OpenMZ's performance with seL4's performance, but a comparison with the MultiZone

Security kernel would be more interesting since it implements the same API as OpenMZ.

Bibliography

[1] Moritz Lipp et al. “Meltdown”. In: *arXiv preprint arXiv:1801.01207* (2018).

[2] Paul Kocher et al. “Spectre attacks: Exploiting speculative execution”. In: *arXiv preprint arXiv:1801.01203* (2018).

[3] *RISC-V History*. RISC-V International. URL: https://riscv.org/risc-v-history/ (visited on June 2020).

[4] *MultiZone Hex Five Security*. URL: https://hex-five.com (visited on Sept. 2019).

[5] Krste Asanović Andrew Waterman. *RISC-V Instruction Set Manual Volume I: Unprivileged ISA*. RISC-V Foundation. URL: https://content.riscv.org/wp-content/uploads/2017/05/riscv-spec-v2.2.pdf (visited on Sept. 2019).

[6] Krste Asanović Andrew Waterman. *RISC-V Instruction Set Manual Volume II: Privileged Architecture*. RISC-V Foundation. URL: https://content.riscv.org/wp-content/uploads/2017/05/riscv-privileged-v1.10.pdf (visited on Sept. 2019).

[7] Krste Asanović and David A Patterson. “Instruction sets should be free: The case for risc-v”. In: *EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2014-146* (2014).

[8] Mohamed Sabt, Mohammed Achemlal, and Abdelmadjid Bouabdallah. “Trusted execution environment: what it is, and what it is not”. In: *2015 IEEE Trustcom/BigDataSE/ISPA*. Vol. 1. IEEE. 2015, pp. 57–64.

[9] *ARM Security Technology*. ARM Limited. URL: infocenter.arm.com/help/topic/com.arm.doc.prd29-genc-009492c/PRD29-GENC-009492C_trustzone_security_whitepaper.pdf (visited on Feb. 2020).

[10] *OP-TEE Documentation*. Website. Linaro Revision. URL: https://optee.readthedocs.io (visited on Sept. 2019).

[11] Victor Costan and Srinivas Devadas. “Intel SGX Explained”. In: *IACR Cryptology ePrint Archive* (2016), pp. 1–118.

[12] Dayeol Lee et al. “Keystone: An Open Framework for Architecting Trusted Execution Environments”. In: *EuroSys '20* (2020), pp. 1–16.

[13] *MultiZone Security SDK for RISC-V*. URL: `https://github.com/hex-five/multizone-sdk` (visited on Sept. 2019).

[14] *MultiZone API*. URL: `https://github.com/hex-five/multizone-api` (visited on Sept. 2019).

[15] James P Anderson. *Computer security technology planning study*. Tech. rep. ANDERSON (JAMES P) and CO FORT WASHINGTON PA FORT WASHINGTON, 1972.

[16] Stephen Smalley, Chris Vance, and Wayne Salamon. “Implementing SELinux as a Linux security module”. In: *NAI Labs Report* 1.43 (2001), p. 139.

[17] *Windows Kernel-Mode Security Reference Monitor - Windows drivers*. URL: `https://docs.microsoft.com/en-us/windows-hardware/drivers/kernel/windows-kernel-mode-security-reference-monitor` (visited on Apr. 2020).

[18] John M Rushby. *Design and verification of secure systems*. Vol. 15. 5. ACM, 1981.

[19] John M Rushby. “Proof of separability a verification technique for a class of security kernels”. In: *International Symposium on Programming*. Springer. 1982, pp. 352–367.

[20] Constance L. Heitmeyer et al. “Applying formal methods to a certifiably secure software system”. In: *IEEE Transactions on Software Engineering* 34.1 (2008), pp. 82–98.

[21] Toby Murray et al. “seL4: From general purpose to a proof of information flow enforcement”. In: *Proceedings - IEEE Symposium on Security and Privacy* (2013), pp. 415–429.

[22] Raymond J. Richards. “Design and verification of microprocessor systems for high-assurance applications”. In: *Modeling and Security Analysis of a Commercial Real-Time Operating System Kernel* (2010), pp. 301–322.

[23] *Common Criteria for Information Technology Security Evaluation*. Common Criteria, 2017.

[24] Jochen Liedtke. “On micro-kernel construction”. In: *ACM SIGOPS Operating Systems Review* 29.5 (1995), pp. 237–250.

[25] Gernot Heiser et al. “seL4: Formal verification of an OS kernel”. In: *Proceedings of the ACM SIGOPS 22nd symposium on Operating systems principles*. ACM. 2009, pp. 207–220.

[26] Michael Schwarz et al. “Keydrown: Eliminating software-based keystroke timing side-channel attacks”. In: *Network and Distributed Systems Security (NDSS) Symposium* (2018).

[27] Jo Van Bulck et al. “FORESHADOW: Extracting the Keys to the Intel SGX Kingdom with Transient Out-of-Order Execution”. In: *Proceedings of the 27th USENIX Security Symposium* (2018), pp. 991–1008.

[28] Ruan De Clercq et al. “Secure interrupts on low-end microcontrollers”. In: *2014 IEEE 25th International Conference on Application-Specific Systems, Architectures and Processors*. IEEE. 2014, pp. 147–152.

[29] *Integer Divide Optimization*. Website. Nullstone Corporation. URL: `https://www.nullstone.com/htmls/category/divide.htm` (visited on Oct. 2019).

[30] Xuejun Yang et al. “Finding and understanding bugs in C compilers”. In: *ACM SIGPLAN Notices*. Vol. 46. 6. ACM. 2011, pp. 283–294.

[31] Xavier Leroy et al. “The CompCert verified compiler”. In: *Documentation and user’s manual. INRIA Paris-Rocquencourt* 53 (2012).

[32] *cloc - count lines of code*. URL: `https://github.com/AlDanial/cloc` (visited on Feb. 2020).

[33] *CompCert - The verified C compiler*. URL: `https://github.com/AbsInt/CompCert` (visited on Nov. 2019).

[34] *RISC-V GNU Compiler Toolchain*. URL: `https://github.com/riscv/riscv-gnu-toolchain` (visited on Oct. 2019).

[35] *HiFive1 Rev B*. URL: `https://www.sifive.com/boards/hifive1-rev-b` (visited on Sept. 2019).

[36] *MultiZone Security for RISC-V*. URL: `https://hex-five.com/multizone-security-sdk/` (visited on Sept. 2019).

[37] *seL4 What is Proved and What is Assumed*. URL: `https://sel4.systems/Info/FAQ/proof.pml` (visited on May 2020).

[38] *HiFive Unleashed*. URL: `https://www.sifive.com/boards/hifive1-unleashed` (visited on Oct. 2019).

[39] *seL4 Performance*. URL: `https://sel4.systems/About/Performance/home.pml` (visited on Apr. 2020).

[40] Gernot Heiser. *How to (and how not to) user seL4 IPC*. URL: https://microkerneldude.wordpress.com/2019/03/07/how-to-and-how-not-to-use-sel4-ipc/ (visited on Apr. 2020).

Appendix A

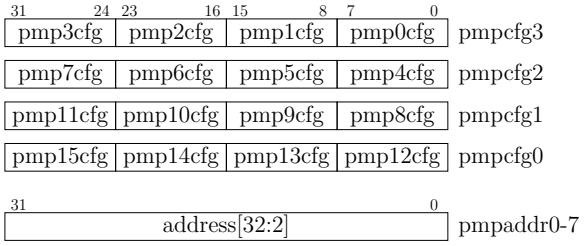
RISC-V

Name	Description
M	integer multiplication and division
A	atomic instructions
F	single-precision floating-point
D	double-precision floating-point
Q	quad-precision floating-point
C	compressed instructions
L	decimal floating-point
B	bit manipulation
J	dynamically translated languages
T	transactional memory
P	packed-SIMD instructions
V	vector operations
N	user-level interrupts
Counters	counter registers
Ztso	total store ordering
Zam	misaligned atomics
Zifencei	instruction fetch fence
Zicsr	control and status registers
G	short for IMAFDZifence_Zicsr

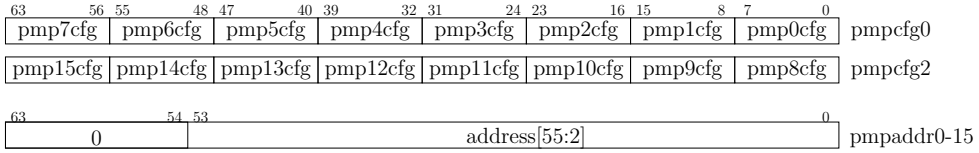
Table A.1: RISC-V extensions.

register name	symbolic name	description	owner
x0	zero	hard-wired zero	-
x1	ra	return address	caller
x2	sp	stack pointer	callee
x3	gp	global pointer	-
x4	tp	thread pointer	-
x5	t0	temporary / alternate link register	caller
x6-7	t1-2	temporaries	caller
x8	s0/fp	saved register / frame pointer	callee
x9	s1	saved register	callee
x10-11	a0-1	function arguments / return values	caller
x12-17	a2-7	function arguments	caller
x18-27	s2-11	saved registers	callee
x28-31	t3-6	temporaries	caller
pc	pc	program counter	-

Table A.2: RISC-V base registers.



(a) RV32



(b) RV64

Figure A.1: The layout of the PMP configuration and the PMP address registers.

pmpaddr$i$ format	alignment and size
yyyy...yyyy0	2^3-byte
yyyy...yyy01	2^4-byte
yyyy...yy011	2^5-byte
yyyy...y0111	2^6-byte
...	...
yy01...11111	2^x-byte
y011...11111	2^{x+1}-byte
0111...11111	2^{x+2}-byte
1111...11111	2^{x+3}-byte

Table A.3: NAPOT encoding in PMP address for RVxI.

Appendix B

MultiZone API

`void ECALL_YIELD()` Indicates to the nanoKernel scheduler that the Zone has nothing pressing to do and causes the nanoKernel to immediately move to the next Zone in context.

`void ECALL_WFI()` Unprivileged implementation of the Wait for Interrupt instruction WFI. The Wait for Interrupt instruction provides a hint to the scheduler that the current zone can be paused until an interrupt might need servicing. If all zones are waiting for interrupt, the scheduler puts the core in a suspended low power state. Note that `ECALL_WFI()` is just a hint: a legal implementation is to implement `ECALL_WFI()` as an `ECALL_YIELD()`, in which case the core will never enter the low power state.

`int ECALL_SEND([Zone #], [0-3][Int])` Send transmits a message from the current zone to the `[Zone #]`; the message size is an array of `[4]` integers and the nanoKernel manages transmission with no shared memory. The value returned is 1 if the receiving mailbox is empty and transmission successful or 0 if the receiving mailbox is full and transmission was blocked.

`int ECALL_RECV([Zone #], [0-3][Int])` Checks the mailbox of the current Zone for a message from the listed `[Zone #]`, if there is a new message the it returns 1, if no new message it returns 0. The value of the message is copied into the array structure provided.

`void ECALL_CSRS_MIE()` Secure user-mode emulation of the Machine Status Register (mstatus) MIE bit. Enables all interrupts (PLIC + CLINT) mapped to the zone, including the soft timer (trap 0x3). The operation is atomic with respect to the context of the zone.

void ECALL_CSRC_MIE() Secure user-mode emulation of the Machine Status Register (mstatus) MIE bit. Disables all interrupts (PLIC + CLINT) mapped to the zone, including the soft timer (trap 0x3). The operation is atomic with respect to the context of the zone.

void ECALL_TRP_VECT([Exception #], [Handler]) Registers a handler against a trap generated by unauthorized instructions; the **[Exception #]**s are defined in the RISC-V Privileged Architectures definition V1.1, Table 3.6 Interrupt 0 types.

void ECALL_IRQ_VECT([Interrupt #], [Handler]) Registers a handler for an interrupt that has been assigned to a Zone. When an interrupt occurs, the nanoKernel will immediately pull the zone assigned to that interrupt into context and execute the registered interrupt handler.

void ECALL_CSRW_MTIMECMP() Secure user-mode emulation of the M-mode timer compare register (mtimecmp). It causes a trap 0x3 exception when the mtime register contains a value greater than or equal to the value assigned. Each zone has its own secure instance of timer and trap handler. Per RISC-V specs, this is a one-shot timer: once set, it will execute its callback function only once. Note that mtime and mtimecmp size is 64-bit even on rv32 architecture. Registering the trap 0x3 handler sets the value of mtimecmp to zero to prevent spurious interrupts. If the timer is set, but no handler is registered, the exception is ignored.

Int64 ECALL_CSRR_MTIME() Returns MTIME to a variable in a zone, MTIME is a privileged register normally only available in M-mode.

Int64 ECALL_CSRR_MCYCLE() Returns MCYCLE to a variable in a zone, MCYCLE is a privileged register normally only available in M-mode.

Int64 ECALL_CSRR_MINSTR() Returns MINSTR to a variable in a zone, MINSTR is a privileged register normally only available in M-mode.

Int64 ECALL_CSRR_MHPMC3() Returns MHPMC3 to a variable in a zone, MHPMC3 is a privileged register normally only available in M-mode.

Int64 ECALL_CSRR_MHPMC4() Returns MHPMC4 to a variable in a zone, MHPMC4 is a privileged register normally only available in M-mode.

Int64 ECALL_CSRR_MISA() Returns MISA to a variable in a zone, MISA is a privileged register normally only available in M-mode.

Int64 ECALL_CSRR_MVENDID() Returns MVENDID to a variable in a zone, MVENDID is a privileged register normally only available in M-mode.

Int64 ECALL_CSRR_MARCHID() Returns MARCHID to a variable in a zone, MARCHID is a privileged register normally only available in M-mode.

Int64 ECALL_CSRR_MIMPID() Returns MIMPID to a variable in a zone, MIMPID is a privileged register normally only available in M-mode.

Int64 ECALL_CSRR_MHARTID() Returns MHARTID to a variable in a zone, MHARTID is a privileged register normally only available in M-mode.

TRITA -EECS-EX-2020:481